

k-playbook

Guardrails & Workflow für AI-gestützte Entwicklung

Vier Säulen:

1. **Enforcement** bestehender Vorgaben (Styleguides, Doku, Konventionen)
2. **Onboarding** – Doku einmalig indexieren & für KI vorbereiten
3. **External Knowledge** – API-/Lib-Doku + Stolperfallen via RAG
4. **Umsetzung** – geplante Änderungen strukturiert & sicher erzeugen
(Task-Pipeline, Git-Diff-Protokoll, Code-Review, Checks, Deploy)

Ziel: Ein wiederverwendbares Set aus Skills, Commands & Templates, das in beliebigen Repos „installiert“ werden kann (siehe `install.md`).

Einordnung

Ich stelle in dieser Session **mein k-playbook** vor — ein Set aus Regeln, Commands und Skills,
das ich über die **letzten Jahre** aus konkreten Projekten heraus entwickelt und iterativ verfeinert habe.

- **Allgemein genug**, um in ganz unterschiedlichen Projekten zu funktionieren.
- **Leicht anpassbar** — projektspezifische Teile (Guidelines, Checks, Reviews) liegen im jeweiligen Repo, das Playbook selbst bleibt schlank.
- Ursprünglich für **Claude Code** geschrieben, funktioniert aber genauso gut mit **OpenCode**.

(Portabilität ist bewusst mitgedacht — `AGENTS.md`, Slash-Commands und Skill-Struktur sind kompatibel.)

Kein akademisches Framework — sondern gewachsene Praxis, die aus dem Alltag

Motivation

Bei **großen Projekten** — übernommenen wie eigenen — entstehen drei Kernprobleme parallel:

A · Mensch verliert den Überblick

- Struktur, Konventionen und implizites Wissen sind nur teilweise dokumentiert.
- Regeln existieren oft nur „in Köpfen“ – Neuzugänge (Mensch *oder* KI) brechen sie unwissentlich.
- Es fehlt eine Basis, um später **verbindliche Regeln zu definieren** und deren Einhaltung zu erzwingen.

B · KI arbeitet ineffizient und unzuverlässig

- Ohne Projekt-Kontext liest die KI in jeder Session dieselben Dateien neu → **Token-Verschwendung**.

Überblick – Die vier Säulen

Säule	Frage	Ergebnis
1 · Enforcement	Wie erzwingen wir bestehende Vorgaben?	Global + projekt-lokale Regeln, <code>k-enforcement</code> -Skill, Checks
2 · Session-Memory	Wie bekommt die AI Projekt-Kontext?	Indexierte Doku, <code>AGENTS.md</code> , Skills
3 · External RAG	Wie kennt die AI unsere Libs & deren Fallen?	Recherche-Playbook + lokaler RAG-Store
4 · Umsetzung	Wie erzeugen wir Änderungen strukturiert & sicher?	Task-Pipeline (create → review → run → code-review), Git-Diff-Protokoll, Deploy-Gate

Säule 1 – Enforcement bestehender Vorgaben

Fokus: KI-gestütztes Enforcement

Zusätzlich zu den klassischen Enforcement-Ebenen (Editor/LSP/Formatter, Pre-commit-Hooks & Commit-Konventionen, CI/CD mit Quality-Gates & Policy-as-Code) **beschäftigen wir uns hier primär mit den Ebenen, bei denen KI und Enforcement sich gegenseitig unterstützen** — also Regeln, die die KI aktiv befolgen muss, und Prüfungen, die durch KI erst effizient möglich werden.

Ablage der Regeln – global + projektspezifisch

Wir nutzen **nicht** `AGENTS.md` als primäre Regelquelle, sondern eine zweistufige Ablage:

- **Global** (im k-playbook: `enforcement/`)
Regeln, die für **alle** Projekte gelten: Dokumentations-Konventionen, allgemeine Do/Don'ts, generelle Arbeitsweise.
- **Projekt-lokal** (Verzeichnis im Repo, z.B. `guidelines/` oder `styleguides/`)
Projekt-spezifische Styleguides, Naming-Conventions, Architektur-Entscheidungen.

`AGENTS.md` verweist nur auf diese beiden Quellen (Pointer statt Inhalt) — so bleibt es kurz und wartbar, und die Regeln sind an einem einzigen Ort.

Umgang mit Legacy-Code: Beide Ablageorte müssen berücksichtigen, dass Bestandscode die neuen Regeln oft (noch) verletzt.

Der Skill **k-enforcement**

Zentraler Baustein: Der Skill **k-enforcement**

- Prüft **laufend während der Erstellung**, dass die Regeln aus global + projekt-lokal tatsächlich eingehalten werden.
- Kein einmaliger Kontext-Push zu Beginn, sondern **kontinuierliche Prüfung** im Arbeitsprozess.
- Ergänzt (nicht ersetzt) die späteren Kontrollen durch `/k-review-loop`, `/k-code-review` und die `checks/`.

Kern-Trennung:

- Skills → Regel-Einhaltung **während** der Erstellung
- Checks → Verifikation **danach** (weil Sollen ≠ Tun, siehe Checks-Abschnitt)

Säule 2 – Doku einmalig anlegen, indexieren & für KI vorbereiten

Warum?

- AI-Sessions starten **ohne Gedächtnis**.
- Vorhandene Doku (docs/ , README, ADRs, Wiki) wird **nicht automatisch gelesen**.
- Wiederholtes „Analysiere den Code neu“ verbrennt Tokens und produziert **inkonsistente Ergebnisse**.

Prinzip: *Doku wird einmal kuratiert → dauerhaft für alle Sessions verfügbar.*

Referenz-Playbook: `ks-ai-session-memory/`

Bausteine

Kontext-Anker

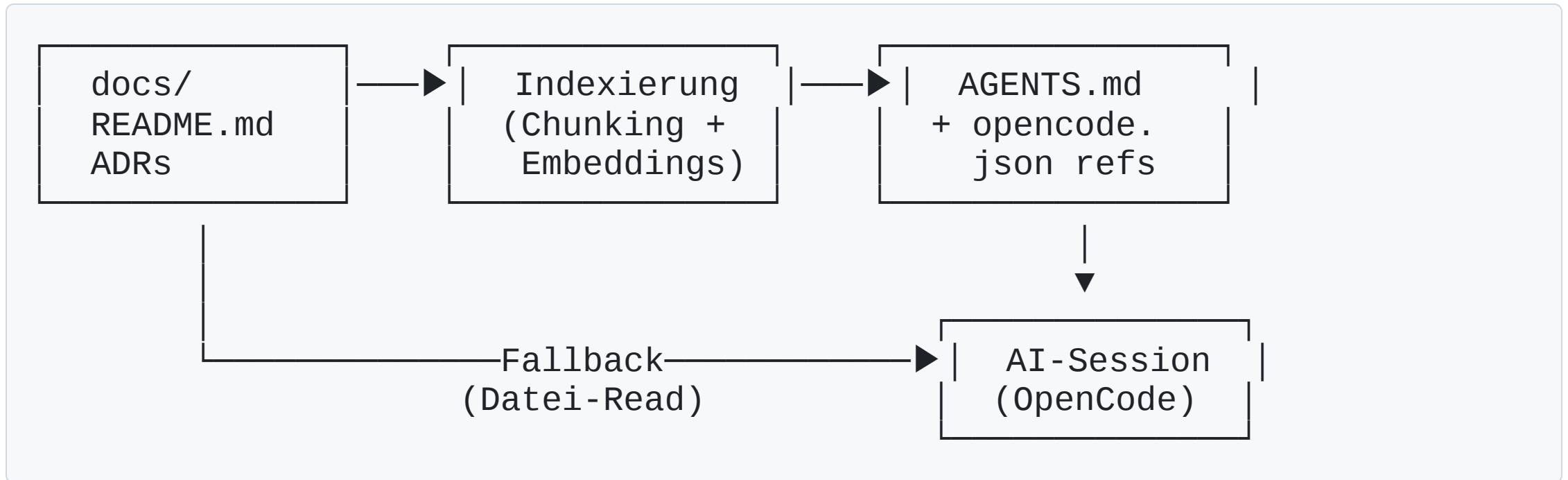
- `AGENTS.md` im Repo-Root
→ wird von OpenCode/Claude Code automatisch geladen
- `opencode.json` mit `instructions[]` + Doc-References
- **Doc-Index** (`docs/README.md`) mit **Keyword-Tags** pro Datei

Doku-Aufbereitung (Doc-as-Code)

- **Chunking** langer Dokumente (Semantic Splitting, header-basiert)
- **Frontmatter-Metadaten** (`tags` , `scope` , `last-reviewed`)
- **Cross-Linking** über relative Pfade
- **ADRs** (Architecture Decision Records) im `docs/adr/` -Format

Optional: Vektor-Index

Säule 2 – Der Workflow



- **Fallback-Ebene:** solange kein RAG steht, reicht `AGENTS.md` + `docs/README.md` .
- **Upgrade-Pfad:** gleiche Doku wird später von RAG-Pipeline konsumiert.

Säule 3 – Externe Doku (APIs / Libs)

Problem

- LLMs kennen Libs oft nur bis zum **Training-Cutoff**.
- Breaking Changes, deprecated APIs, „Version-Pinning-Blindness“.
- **Stolperfallen** (Rate-Limits, Auth-Quirks, Non-Idempotenz, Timezone-Bugs) stehen selten in offizieller Doku prominent.
- Suche im Web ist teuer, laut und schwer reproduzierbar.

Ziel: *Kuratiertes, versionsgenaues Wissen über jede genutzte Library — mit Fokus auf **Pitfalls**, nicht Copy-Paste-Snippets.*

Recherche-Playbook

1. Dependencies erfassen

- Beispielsweise aus `pyproject.toml`, `package.json`, `go.mod`, `Cargo.toml` — oder anderen projektypischen Manifesten
- Version pro Dependency festhalten (**SBOM-Ansatz**)

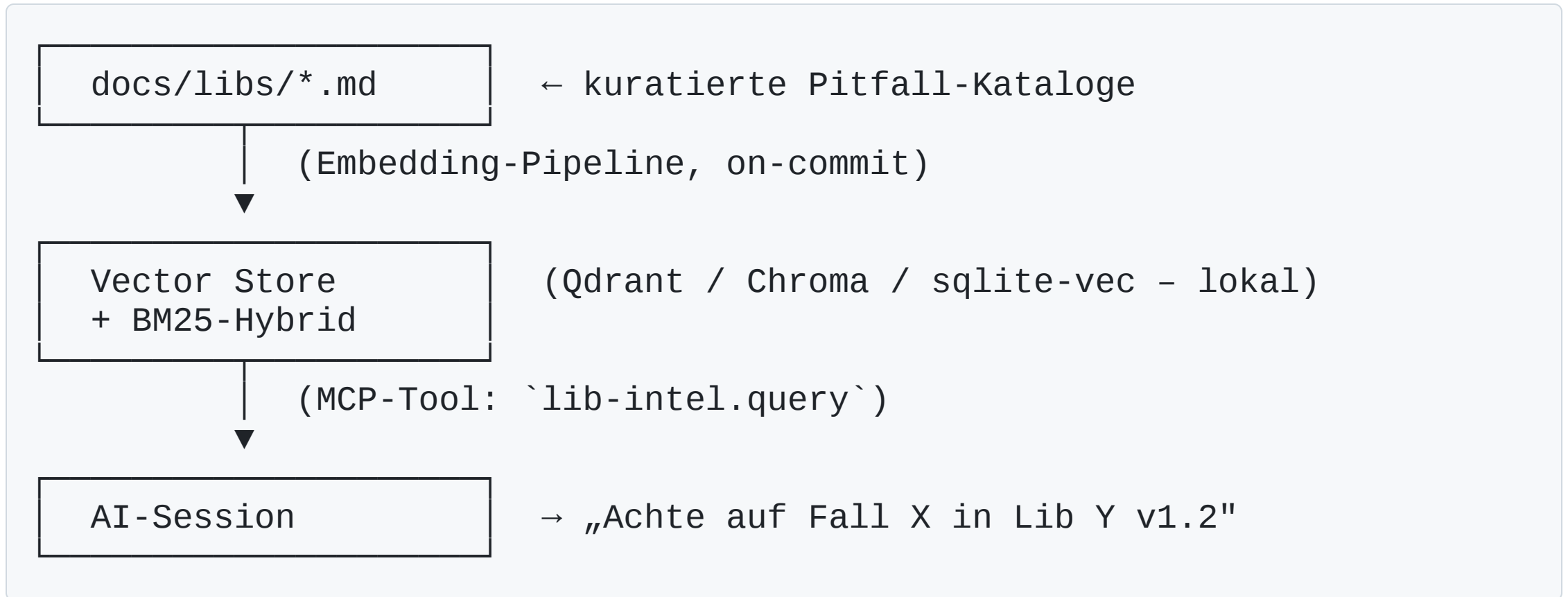
2. Recherche-Quellen

- Offizielle Docs, Changelog, GitHub Issues (Labels `bug`, `gotcha`)
- Stack Overflow (Top-Voted Gotchas)
- **DevDocs.io**, **Context7-MCP**, `docs.rs`, `pkg.go.dev`
- Security-DBs: **OSV**, **GHSA**, NVD

3. Extraktion

- Pro Lib → strukturierte Markdown-Datei:

Säule 3 – RAG-Architektur (minimal)



Design-Prinzipien:

- **Hybrid Retrieval** (BM25 + Dense) für Fachbegriffe & Codenamen

Säule 4 – Umsetzung

Änderungen strukturiert & nachvollziehbar erzeugen

Die ersten drei Säulen sorgen dafür, dass **Regeln, Projekt-Kontext und externes Wissen** verfügbar sind.

Säule 4 nutzt das, um **konkrete Änderungen am Code kontrolliert durchzuführen** — von der Idee bis zum Deploy-Gate.

Bausteine (im Folgenden je ein eigener Abschnitt):

Baustein	Zweck
Tasks (/k-create-task)	Aus Gespräch → kuratierte, self-contained Task-Datei
Reviews (/k-review-loop)	Perspektiv-Umkehr auf den Task, bevor Code entsteht
Ausführung (/k-run)	Sequentielle Umsetzung, Git-protokolliert

Tasks

Warum ein eigenes Task-Format?

Übliche Praxis: Aufgabe direkt im Chat besprechen → Agent los.

Problem: Der ausführende Agent muss

- **selbst suchen**, welche Dateien / Docs relevant sind → Tokens verbrannt, Kontext schwillt an.
- **raten**, wenn die Suche zu aufwändig wird → Halluzinationen, Anti-Patterns.
- **improvisieren**, was das eigentliche Ziel („Intent“) ist → Sackgassen, Rework.

Lösung im k-playbook: Der Task ist ein **kuratiertes, self-contained Dokument** — erzeugt aus dem Gespräch, bevor der ausführende Agent überhaupt startet.

/k-task-create – was der Command tut

Aus der laufenden Konversation wird eine strukturierte Task-Datei erzeugt:

- **Nummerierung** automatisch (`tasks/` + `tasks/old/` gescannt → nächste freie Nummer, `014-audiosocket-server.md`)
- **## Intent** — 1–2 Sätze Zielrahmen + 2–5 Erfolgskriterien
(bei Task-Serien nur im **letzten** File — dient als Anker für Alignment)
- **## Referenzen** — alle im Gespräch erwähnten relevanten Dateien/Docs mit Begründung
- **## Tools** — nur **zusätzliche** Tools über Standard-Set hinaus (MCP, spezielle Permissions)
- **## Ziel** / **## Kontext** / **## Zu bauen** — was, warum, konkret welche Artefakte
- **Confirm-Loop** — Draft wird gezeigt, User bestätigt, dann erst gespeichert

Ergebnis: Der Task enthält bereits alles, was der ausführende Agent (und der

Wirkung: weniger suchen, nicht raten

Token-Effizienz

- Referenzen sind kuratiert → Agent liest gezielt, nicht explorativ.
- Kontext wird **einmal** vom Menschen zusammengestellt statt **jedes Mal** von der KI erneut erschlossen.

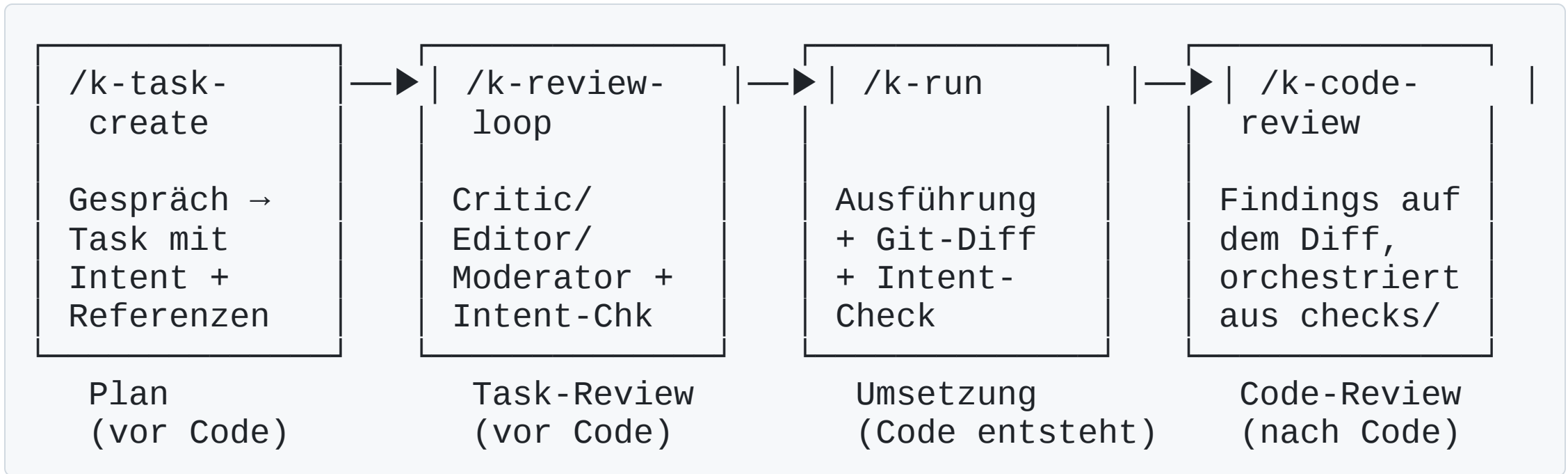
Weniger Halluzinationen

- Der klassische Auslöser: „Suche wäre zu teuer → ich rate mal plausibel.“
- Wenn die Quelle direkt im Task steht, entfällt der Anreiz zur Improvisation.

Weniger Verrennen

- **Intent** gibt harten Erfolgsmaßstab → Agent kann Sackgassen frühzeitig erkennen.
- **Zu bauen** grenzt den Scope ab → keine Feature-Creep-Rabbit-Holes.

Die Pipeline: create → review → run → code-review



Zwei Reviews mit unterschiedlichem Gegenstand — bewusst unterschiedlich benannt:

- **Review** (/k-review-loop) prüft den **Task** — bevor Code entsteht.
Frage: „Erreicht der Plan sein Intent?“

Reviews

Der übliche Ansatz (und warum er teuer ist)

Viele AI-Review-Setups setzen auf:

- **Mehrere LLMs parallel** (Claude + GPT + Gemini ...) → Kosten $\times$ N, Latenz $\times$ N
- **Verschiedene Personas** („Security-Reviewer“, „Senior-Dev“, „QA“) auf demselben Prompt
- **Consensus-Voting** über die Ergebnisse

Problem: Die eigentliche Motivation ist ein **Perspektivwechsel** auf den Code — aber Personas und Modellwechsel liefern das nur unzureichend.

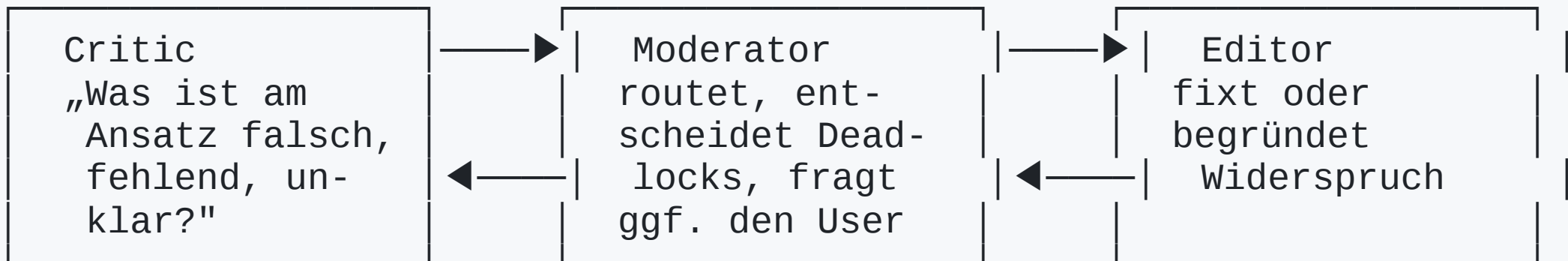
Die Modelle sehen dieselbe Aufgabe aus demselben Blickwinkel — nur mit anderem „Kostüm“.

Der k-playbook-Ansatz: Perspektivwechsel durch Aufgabenumkehr

Grundidee: Nicht das *Modell* wechseln, sondern die **Aufgabenstellung drehen**.
Dadurch wird der Blickwinkel **strukturell** anders — auch mit demselben (günstigen) LLM.

Konkret umgesetzt in `/k-review-loop`

Review von **Task-/Instruction-Dateien vor der Ausführung** — strukturierter Dialog:



Design-Details von /k-review-loop

- **Drei Rollen, ein Modell:** Critic · Editor · Moderator — als separate Subagent-Turns, jede mit eigenem Prompt/Blickwinkel.
- **High-Level-Review:** Prüft **Ansatz, Widersprüche, fehlende Constraints** — nicht Style oder Details. Der ausführende Agent füllt Details.
- **Kategorien:** FEHLER (blockierend) · WARNUNG · FEHLEND — Moderator filtert, nur FEHLER gehen automatisch in die Editor-Runde.
- **Deadlock-Handling:** Bis zu 5 Runden Critic ↔ Editor; wiederholt sich das Argument → Moderator entscheidet und dokumentiert.
- **Intent als externer Anker:** Inline im Task oder als Datei-Referenz (`[requirements.md](...)`). Der finale Alignment-Check fragt nur: „*erreicht der Task sein Intent, ja/nein?*“
- **Nachvollziehbarkeit:** Discussion-Log wird an die Task-Datei angehängt

Weitere Perspektiv-Umkehrungen (Muster)

Dasselbe Prinzip lässt sich auf weitere Phasen anwenden:

Rolle A (produzierend)	Rolle B (invertierend)
„Schreibe diesen Task“	„ Erreicht der Task sein Intent?“ (← <code>/k-review-loop</code>)
„Implementiere Feature X“	„Welche Tests würden diese Implementierung <i>widerlegen</i> ?“
„Schreibe diese Funktion“	„Wie würdest du sie missbrauchen / zum Absturz bringen?“
„Refactor dieses Moduls“	„Welche Regressionen kann dieser Refactor auslösen?“
„Schreibe die Doku dazu“	„Verstehe nur aus der Doku, was der Code tut — was fehlt?“

Warum das billiger *und* effizienter ist

- **Ein Modell reicht** — auch ein kleineres/günstigeres.
- **Kein Consensus-Overhead** — zwei komplementäre Turns statt N paralleler.
- **Strukturelle Diversität** statt oberflächlicher (Kostüm-)Diversität.
- **Reproduzierbar**: Die Umkehrung ist im Prompt-Template festgeschrieben, nicht im „Bauchgefühl“ eines Personals.
- **Composable**: Mehrere Umkehrungen lassen sich verketteten (Vorschlag → Prüfung → Gegenprüfung).

Umsetzung im k-playbook

- `/k-review-loop <path>` (*vorhanden*)
 - Pre-Execution-Review von Task-/Instruction-Dateien
 - Critic ↔ Editor ↔ Moderator, bis zu 5 Runden
 - **Intent-Alignment** als finaler Check (Inline-Text oder Datei-Referenz)
 - Discussion-Log wird in die Task-Datei geschrieben (Audit-Trail)
- **Skill** `ks-review-flip` (*geplant*)
 - überträgt das Muster auf **weitere Phasen** (Implementation, Refactor, Doku, Bug-Fix)
 - liefert Prompt-Templates für die häufigsten Rollen-Umkehrungen
 - Verkettung mehrerer Umkehrungen (Producer → Inverter → Judge)
- **Integration mit Säule 1:** Wiederkehrende Findings aus Review-Logs
→ neue Do/Don't-Einträge in `AGENTS.md` (Regeln wachsen aus Beobachtung).

Säule 3 – Fachbegriffe im Überblick

- **RAG** – Retrieval-Augmented Generation
- **Embeddings / Dense Retrieval / Sparse Retrieval (BM25)**
- **Chunking-Strategien**: Fixed-size, Recursive, Semantic, Late-Chunking
- **Re-Ranking** (Cross-Encoder, `bge-reranker`)
- **MCP** – Model Context Protocol (Tools & Resources für die AI)
- **Context7** – Live-Doku-MCP-Server für Libraries
- **SBOM** – Software Bill of Materials (Dependency-Inventar)
- **OSV / GHSA** – Vulnerability-Datenbanken

Ausführung (/k-run)

Was /k-run tut

Führt eine oder mehrere Task-Dateien sequentiell aus — mit strikter Absicherung drumherum:

- **Reihenfolge nach Nummer** (013-... vor 014-...), **nie parallel** (zwei Agents dürfen nicht gleichzeitig Code ändern)
- **Tools-Upfront-Anzeige** — alle zusätzlichen Tools/Permissions aus allen Tasks werden **vor Start** gesammelt und dem User angezeigt (keine Nachfragen mitten im Lauf)
- **Klärung im Main-Context** (Step 2b) — offene Fragen werden **vor** der Delegation an den Sub-Agent gestellt; der Sub-Agent darf explizit nicht raten
- **Sub-Agent-Isolation** — jede Task-Ausführung in eigenem Sub-Agent, Blocker

Git-Absicherung & Protokoll (Kern-Vorteile)

Der Command nutzt Git als **Wahrheits-Anker** — jeder Task hinterlässt einen nachvollziehbaren Fußabdruck:

Ablauf pro Task:

1. Vor Start: `BASELINE_HASH = git rev-parse HEAD` festhalten
2. Optional: `before:` -Kommando aus `CLAUDE.md` (z.B. `make sichern` als Backup)
3. Task wird ausgeführt
4. Nach Erfolg: `git diff --stat` + `git diff $BASELINE_HASH` erzeugen
5. Diff wird **direkt in die Task-Datei** unter `## Ausführung` geschrieben
6. Optional: `after:` -Kommando (z.B. erneutes Backup / Deploy-Trigger)

Warum das entscheidend ist:

- **Nachweis „nur was zu tun war, wurde getan“** — der Diff ist per Konstruktion die

Code-Review – Ansatz

Prinzip: Für Code-Reviews werden **keine eigenen** Skills/Commands gebaut, sondern **bewährte fremde Sammlungen** eingebunden und orchestriert.

- Review-Regeln sind **projekt- und sprachspezifisch** → keine allgemeingültige Lösung sinnvoll.
- Ablage der projektspezifischen Definitionen im Verzeichnis `reviews/` im Projekt-Repo.
- Das k-playbook liefert nur die **Orchestrierung**.

Die konkrete Ausgestaltung (Auswahl-Logik, Format der Review-Dateien, unterstützte Sammlungen) ist umfangreich und wird an dieser Stelle bewusst ausgespart — Detailkonzept folgt in einem eigenen Kapitel.

Auswertung & Umsetzung

Zwei Bausteine reichen für diese Präsentation aus:

1 · Auswertungs-Command

- Startet eine Review-Session und geht **jeden Findpunkt der Reihe nach** durch.
- Jeder Punkt wird **sauber geprüft** (Kontext, Relevanz, Schweregrad) und dem User **strukturiert vorgestellt**.
- Keine Batch-Ausgabe von 200 Findings am Stück — sondern moderierter Durchgang.

2 · Anschluss nach der Besprechung

Nach dem Durchgang werden die zu adressierenden Punkte auf zwei Wege verteilt:

- **Einfache Sachen** → Task-Sammlung erzeugen und über den **üblichen Workflow** lösen

Checks

Was ist ein „Check“?

Neben den generischen Playbook-Bausteinen hat jedes Projekt ein **projekt-spezifisches Verzeichnis** `checks/` — eine Sammlung aus `.md`-Dateien, Shell-Skripten und Kommandos, die je nach Anforderung aufgerufen werden.

Spannweite:

- **Einfach & schnell:** Unit-Tests starten, Smoke-Tests, Lint-Runs
- **Aufwändiger:** Prüfungen, dass vorher festgelegte **Erfordernisse tatsächlich eingehalten** werden
 - z.B. Doku ↔ Code-Konsistenz (in **beide** Richtungen)
 - Doku widerspricht sich nicht selbst

Ausführungs-Modi

Checks unterscheiden sich stark in Laufzeit — manche sind Sekunden, manche Minuten oder länger.

Deshalb wird beim Aufruf **immer der Modus abgefragt**:

- **schnell** — nur die günstigen Checks (z.B. Unit-Tests, statische Prüfungen)
- **umfassend** — komplettes Set inkl. langlaufender Prüfungen (Doku-Konsistenz, End-to-End)
- **Auswahl** — gezielte Teilmenge (z.B. nur die Checks, die zum aktuellen Change passen)

Zusätzlich lässt sich beim Aufruf **explizit angeben**, welche Checks laufen sollen — für gezielte Deep-Checks eines bestimmten Themas.

Ergebnis-Aggregation: Die Ergebnisse aller gelaufenen Checks werden strukturiert vorgestellt

Pre-Deploy-Check

Absicherung vor dem Deploy

Bevor Code nach Dev / Prod geht, wird ein **orchestrierter Check** ausgelöst — **Checks + Reviews gemeinsam**, aber mit **anderer Filterung** als im normalen Alltag.

Ablauf:

1. **Checks komplett aufrufen** (umfassender Modus)
2. **Reviews starten**
3. Ergebnisse strikt gefiltert präsentieren (siehe nächste Slide)

Ziel: nichts wird veröffentlicht, was nachweislich blockiert — aber der Prozess erstickt nicht in Verbesserungsvorschlägen.

Der Deploy selbst ist **nicht Teil** des k-playbook — er läuft weiterhin über die projekt-

Pre-Deploy-Filter: nur Show-Stopper

Beide Quellen — Checks und Reviews — liefern normalerweise **viele Findings**.

Vor einem Deploy interessieren aber **nur die harten**:

Reviews im Pre-Deploy-Modus:

-  Tatsächlich **entscheidende Fehler** werden aufgelistet
-  Verbesserungsvorschläge, Style, „Nice-to-Have“ **fallen weg**

Checks im Pre-Deploy-Modus:

-  Alles, was gegen eine Veröffentlichung **spricht** oder sie **verhindert**
-  **Keine Verbesserungen** — nur Blocker

Warum diese Härte-Filterung?

- Wenn das Gate mit Style-Diskussionen zugemüllt ist, wird es ignoriert.

Vor größeren Deploys (oder zwischendurch)

Zusätzlich zum blockierenden Gate: **umfassender Check** vor größeren Meilensteinen — mit Fokus auf **Konsistenz**:

- **Doku intern** — widerspricht sich die Doku selbst?
- **Code ↔ Doku (beide Richtungen)**
 - Beschreibt die Doku noch, was der Code tut?
 - Wird alles, was der Code tut, von der Doku erfasst?
- Wiederkehrende Invarianten aus `checks/`

Kopplung zurück zu den Tasks:

Weil zu jedem Task der **Doku-Nachzug** gehört (siehe Tasks-Abschnitt), hat dieser Check überhaupt eine faire Chance — sonst wäre das Delta zwischen Code und Doku nach kurzer Zeit unaufholbar.

- **Governance:** Wer pflegt die Pitfall-Kataloge? Review-Kadenz?
- **Verteilung:** Framework als **Template-Repo**, **degit-Snapshot** oder **Git-Submodule**?
- **Portabilität:** OpenCode-Skills ↔ Claude-Code-Skills ↔ Cursor-Rules ↔ generisches `AGENTS.md` ?
- **Messbarkeit:**
 - Wie messen wir Wirkung? (Anzahl korrigierter Anti-Patterns, weniger Review-Runden, ...)
 - Baseline-Metriken vor/nach Einführung?
- **Datenschutz:** Welche Doku darf in Cloud-Embeddings? Was bleibt strikt lokal?

